

Secure Software Design and Development

Secure Student Management System

Submitted by: Syed Jazib Ali Rizvi, Hussain Ali

Technology Stack: Flask, Flask-SQLAlchemy, Flask-WTF, SQLite, Bootstrap, Docker

Project Type: Secure CRUD Web Application

1. Introduction

The Secure Student Management System is a Flask-based web application designed to manage student records through an authenticated admin dashboard. The application supports admin registration/login, student record creation, record listing, update, and deletion. The final version applies secure software design principles such as password hashing, input validation, CSRF protection, secure session settings, and safe project structure.

2. Objectives

- Develop a functional student management system using Flask.
- Apply secure coding practices in authentication, form handling, and CRUD operations.
- Use validation to reduce invalid or unsafe input.
- Organize all application code inside a dedicated src/ directory.
- Avoid committing secrets, database files, virtual environments, or generated cache files.

3. Application Features

Feature	Description
Admin Registration	Only the first admin can be registered through the app.
Admin Login	Admin credentials are checked using hashed passwords.
Dashboard	Authenticated users can view the student management dashboard.
Add Student	Validated form fields are used to add student records.
Edit Student	Existing records can be updated through a protected route.
Delete Student	Deletion is handled using POST requests with CSRF protection.

4. Secure Design Practices Applied

Security Control	Implementation
Password Hashing	Admin passwords are stored using Werkzeug password hashing instead of plaintext storage.
CSRF Protection	Flask-WTF CSRF protection is enabled globally and applied to login, register, logout, add, edit, and delete forms.
POST-only Delete	Delete operations use POST instead of GET to avoid unsafe state-changing links.
Input Validation	Flask-WTF and WTForms validators enforce required fields, length limits, numeric ranges, email format, and alphabetic name/city input.
Session Security	HTTPOnly, SameSite=Lax, and environment-based Secure cookie settings are configured.
Secret Management	Real .env files are excluded. A .env.example file documents required variables without exposing secrets.
Safe Repository Hygiene	The final repository excludes .git, env, __pycache__, database files, and runtime artifacts.

5. Final Repository Structure

secure-student-management-system/

```
├── src/
│   ├── app.py
│   ├── forms.py
│   ├── create_db.py
│   ├── templates/
│   └── static/
├── docs/
│   └── secure-student-management-system-report.docx
├── screenshots/
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── .env.example
├── README.md
├── PROJECT_NOTES.md
├── RUN_GUIDE.md
└── .gitignore
```

6. Screenshots

Figure: Dashboard

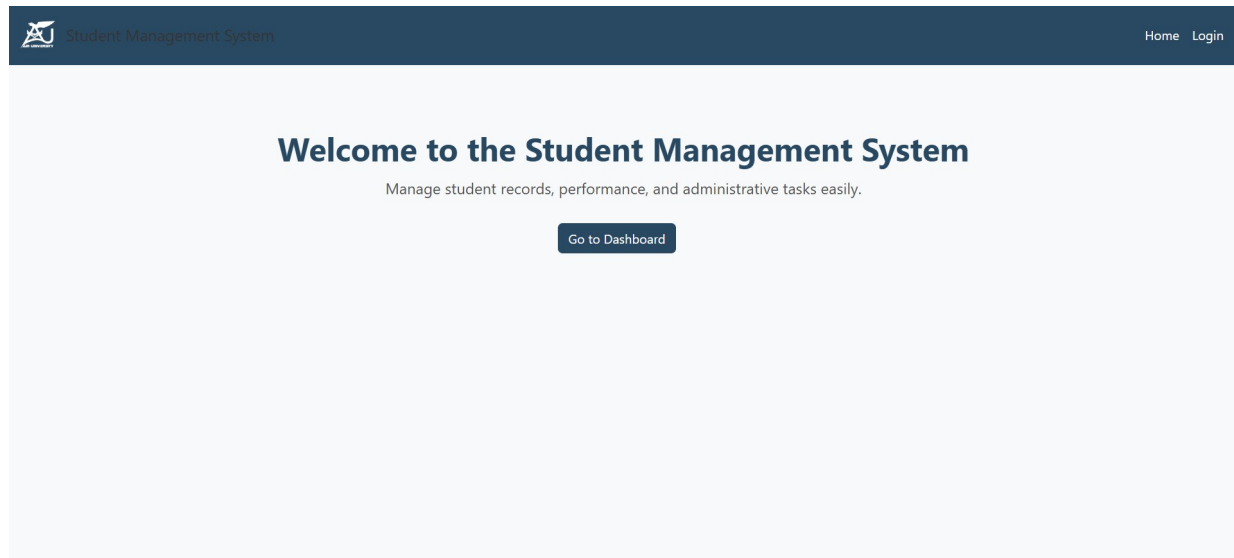


Figure: Code Preview

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dashboard - SMS</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.10.5/font/bootstrap-icons.css">
  <style>
    body {
      background-color: #f8f9fa;
    }

    .navbar {
      background-color: #294861;
    }

    .navbar .nav-link,
    .navbar-brand {
      color: #fff;
      transition: color 0.3s ease;
    }

    .navbar .nav-link:hover {
      color: #f8c146;
    }

    .navbar-toggler {
      border: none;
      font-size: 1.5rem;
      color: white;
      transition: transform 0.3s ease;
    }
  </style>
</head>

```

Figure: Admin Login

Student Management System Home Login

You have been logged out.

Admin Login

Username
admin

Password

Login

Not registered? [Register here.](#)

Figure: Add Student Form

 Student Management System

Home Logout

Add New Student

First Name

Last Name

Age

City

Email

Add Student

Student Records

#	First Name	Last Name	Age	City	Email	Actions
No students found.						

Figure: Student Record Added

 Student Management System

Home Logout

Add New Student

First Name

Last Name

Age

City

Email

Add Student

Student Records

#	First Name	Last Name	Age	City	Email	Actions
1	Hussain	Ali	19	Islamabad	darkvilen1112@gmail.com	EditDelete

Figure: Edit Student Record

Student Management System Home Logout

Update Student Information

First Name
Hussain

Last Name
Ali

Age
19

City
Islamabad

Email
darkvilen1112@gmail.com

Back to Dashboard Update Student

Figure: Duplicate Admin Warning

Student Management System Home Login

Admin already registered. Please log in. X

Admin Login

Username
Password

Login

Not registered? [Register here.](#)

7. Limitations

- The application is a coursework-level secure CRUD system, not a production Student Information System.
- SQLite is used for simplicity; production deployments should use a managed database.
- Role-based access control is not implemented because the application has a single-admin model.
- Additional production hardening such as HTTPS termination, rate limiting, audit logging, and deployment secrets management can be added later.

8. Future Improvements

- Add password confirmation and stronger password complexity checks.
- Add rate limiting for login attempts.
- Add audit logs for create, update, and delete actions.
- Add role-based access control for admin and staff users.
- Add unit tests for forms, routes, and security behavior.
- Move inline CSS into static files for cleaner maintainability.

9. Conclusion

The Secure Student Management System demonstrates how a basic Flask CRUD application can be improved using secure software design and development practices. The final project protects authentication with password hashing, validates user input, applies CSRF protection, changes destructive actions to POST requests, and removes sensitive runtime files from the public repository. These changes make the project more suitable for academic submission and GitHub portfolio presentation.

10. References

1. Flask Documentation: <https://flask.palletsprojects.com/>
2. Flask-WTF Documentation: <https://flask-wtf.readthedocs.io/>
3. SQLAlchemy Documentation: <https://docs.sqlalchemy.org/>
4. Werkzeug Security Helpers: <https://werkzeug.palletsprojects.com/>
5. OWASP Secure Coding Practices Quick Reference Guide.